

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Naskah Soal Praktikum 2

Query Processing and Optimization

Jumat, 25 September 2026

Oleh:

13524089 - Aurelia Jennifer Gunawan

13524098 - Reva Natania Sitohang

IF3140 - Sistem Basis Data
2026

Petunjuk

1. Kerjakan setiap soal praktikum ini dengan baik
2. Diperkenankan untuk menggunakan dokumentasi PostgreSQL resmi
3. Nama database: airport
4. **Dilarang Menyontek**
5. **`docker exec praktikum-db-1 pg_dump -U postgres -d airport -h localhost -p 5432 > [DATABASE_FILE_EXPORTED].sql`**

airport



1. Tiga Teknik

Peter dan Brian tengah terlibat dalam perdebatan panas ketika Stewie tiba di ruang tamu. Keduanya yakin bahwa query buatan mereka masing-masing adalah yang paling efisien untuk mendapatkan data yang diinginkan. Melihat ketegangan yang meningkat, Stewie mencoba menengahi dengan membuat versinya sendiri dari query tersebut, dengan hasil yang sama, berharap dapat meredakan situasi. Sayangnya, usahanya justru memperburuk keadaan, karena kini Stewie pun ikut ngotot membela hasil karyanya sendiri. Bantulah Peter, Brian, dan Stewie menyelesaikan perdebatan mereka dengan melakukan analisis terhadap 3 *query* berikut.

Notes: Gunakan `EXPLAIN ANALYZE` untuk pengerjaan soal ini

Direct Join
<pre> SELECT a.airlinename, ROUND(COUNT(CASE WHEN pd.sex = 'M' THEN 1 END)::NUMERIC / NULLIF(COUNT(*), 0), 4) AS rasio_laki_laki, ROUND(COUNT(CASE WHEN pd.sex = 'F' THEN 1 END)::NUMERIC / NULLIF(COUNT(*), 0), 4) AS rasio_perempuan FROM airline a JOIN flight f ON a.airline_id = f.airline_id JOIN booking b ON f.flight_id = b.flight_id JOIN passengerdetails pd ON b.passenger_id = pd.passenger_id GROUP BY a.airline_id, a.airlinename ORDER BY a.airlinename; </pre>
Common Table Expression (CTE)
<pre> WITH airline_passenger_summary AS (SELECT f.airline_id, COUNT(CASE WHEN pd.sex = 'M' THEN 1 END) AS count_male, COUNT(CASE WHEN pd.sex = 'F' THEN 1 END) AS count_female, </pre>

```

        COUNT(*) AS total_passengers
    FROM flight f
    JOIN booking b ON f.flight_id = b.flight_id
    JOIN passengerdetails pd ON b.passenger_id =
pd.passenger_id
    GROUP BY f.airline_id
)
SELECT
    a.airlinename,
    ROUND(aps.count_male::NUMERIC /
NULLIF(aps.total_passengers, 0), 4) AS rasio_laki_laki,
    ROUND(aps.count_female::NUMERIC /
NULLIF(aps.total_passengers, 0), 4) AS rasio_perempuan
FROM airline a
JOIN airline_passenger_summary aps ON a.airline_id =
aps.airline_id
ORDER BY a.airlinename;

```

Correlated Subquery

```

SELECT
    a.airlinename,
    ROUND(
        (SELECT COUNT(*)
        FROM flight f
        JOIN booking b ON f.flight_id = b.flight_id
        JOIN passengerdetails pd ON b.passenger_id =
pd.passenger_id
        WHERE f.airline_id = a.airline_id AND pd.sex =
'M')::NUMERIC
        /
        NULLIF(
            (SELECT COUNT(*)
            FROM flight f
            JOIN booking b ON f.flight_id = b.flight_id
            WHERE f.airline_id = a.airline_id), 0
        ),
        4
    ) AS rasio_laki_laki,
    ROUND(
        (SELECT COUNT(*)
        FROM flight f
        JOIN booking b ON f.flight_id = b.flight_id
        JOIN passengerdetails pd ON b.passenger_id =
pd.passenger_id
        WHERE f.airline_id = a.airline_id AND pd.sex =
'F')::NUMERIC
        /
        NULLIF(
            (SELECT COUNT(*)
            FROM flight f
            JOIN booking b ON f.flight_id = b.flight_id
            WHERE f.airline_id = a.airline_id), 0
        ),
        4
    ) AS rasio_perempuan
FROM airline a

```

```

4
) AS rasio_perempuan
FROM airline a
ORDER BY a.airlinename;

```

```

QUERY PLAN
-----
Sort (cost=3328.31..3328.46 rows=60 width=81) (actual time=38.490..38.493 rows=60 loops=1)
  Sort Key: a.airlinename
  Sort Method: quicksort Memory: 28kB
  -> HashAggregate (cost=3324.43..3326.53 rows=60 width=81) (actual time=38.399..38.432 rows=60 loops=1)
    Group Key: a.airline_id
    Batches: 1 Memory Usage: 24kB
    -> Hash Join (cost=484.85..2244.43 rows=72000 width=19) (actual time=2.350..28.759 rows=72000 loops=1)
      Hash Cond: (b.passenger_id = pd.passenger_id)
      -> Hash Join (cost=219.35..1789.82 rows=72000 width=21) (actual time=1.133..19.792 rows=72000 loops=1)
        Hash Cond: (f.airline_id = a.airline_id)
        -> Hash Join (cost=217.00..1585.12 rows=72000 width=8) (actual time=1.111..12.911 rows=72000 loops=1)
          Hash Cond: (b.flight_id = f.flight_id)
          -> Seq Scan on booking b (cost=0.00..1179.00 rows=72000 width=8) (actual time=0.002..3.189 rows=72000 loops=1)
          -> Hash (cost=132.00..132.00 rows=6800 width=8) (actual time=1.084..1.084 rows=6800 loops=1)
            Buckets: 8192 Batches: 1 Memory Usage: 330kB
            -> Seq Scan on flight f (cost=0.00..132.00 rows=6800 width=8) (actual time=0.001..0.475 rows=6800 loops=1)
        -> Hash (cost=1.60..1.60 rows=60 width=17) (actual time=0.016..0.016 rows=60 loops=1)
          Buckets: 1024 Batches: 1 Memory Usage: 11kB
          -> Seq Scan on airline a (cost=0.00..1.60 rows=60 width=17) (actual time=0.003..0.006 rows=60 loops=1)
      -> Hash (cost=178.00..178.00 rows=7000 width=6) (actual time=1.198..1.198 rows=7000 loops=1)
        Buckets: 8192 Batches: 1 Memory Usage: 338kB
        -> Seq Scan on passengerdetails pd (cost=0.00..178.00 rows=7000 width=6) (actual time=0.004..0.516 rows=7000 loops=1)
Planning Time: 0.300 ms
Execution Time: 38.622 ms
(24 rows)

Time: 39.438 ms

```

```

QUERY PLAN
-----
Sort (cost=3126.72..3126.87 rows=60 width=77) (actual time=32.544..32.547 rows=60 loops=1)
  Sort Key: a.airlinename
  Sort Method: quicksort Memory: 28kB
  -> Hash Join (cost=3121.68..3124.95 rows=60 width=77) (actual time=32.479..32.516 rows=60 loops=1)
    Hash Cond: (a.airline_id = aps.airline_id)
    -> Seq Scan on airline a (cost=0.00..1.60 rows=60 width=17) (actual time=0.004..0.006 rows=60 loops=1)
    -> Hash (cost=3120.93..3120.93 rows=60 width=28) (actual time=32.463..32.464 rows=60 loops=1)
      Buckets: 1024 Batches: 1 Memory Usage: 12kB
      -> Subquery Scan on aps (cost=3119.73..3120.93 rows=60 width=28) (actual time=32.441..32.451 rows=60 loops=1)
        -> HashAggregate (cost=3119.73..3120.33 rows=60 width=28) (actual time=32.441..32.448 rows=60 loops=1)
          Group Key: f.airline_id
          Batches: 1 Memory Usage: 24kB
          -> Hash Join (cost=482.50..2039.73 rows=72000 width=6) (actual time=2.516..22.646 rows=72000 loops=1)
            Hash Cond: (b.passenger_id = pd.passenger_id)
            -> Hash Join (cost=217.00..1585.12 rows=72000 width=8) (actual time=1.199..13.729 rows=72000 loops=1)
              Hash Cond: (b.flight_id = f.flight_id)
              -> Seq Scan on booking b (cost=0.00..1179.00 rows=72000 width=8) (actual time=0.002..3.060 rows=72000 loops=1)
              -> Hash (cost=132.00..132.00 rows=6800 width=8) (actual time=1.177..1.177 rows=6800 loops=1)
                Buckets: 8192 Batches: 1 Memory Usage: 330kB
                -> Seq Scan on flight f (cost=0.00..132.00 rows=6800 width=8) (actual time=0.002..0.555 rows=6800 loops=1)
            -> Hash (cost=178.00..178.00 rows=7000 width=6) (actual time=1.299..1.300 rows=7000 loops=1)
              Buckets: 8192 Batches: 1 Memory Usage: 338kB
              -> Seq Scan on passengerdetails pd (cost=0.00..178.00 rows=7000 width=6) (actual time=0.003..0.586 rows=7000 loops=1)
Planning Time: 0.292 ms
Execution Time: 32.678 ms
(25 rows)

Time: 33.516 ms

```

```

QUERY PLAN
-----
Sort (cost=394080.58..394080.73 rows=60 width=77) (actual time=1509.621..1509.626 rows=60 loops=1)
  Sort Key: a.airline_id
  Sort Method: quicksort  Memory: 28kB
  -> Seq Scan on airline a (cost=0.00..394078.81 rows=60 width=77) (actual time=49.336..1509.514 rows=60 loops=1)
    SubPlan 1
      -> Aggregate (cost=1761.98..1761.99 rows=1 width=8) (actual time=6.540..6.540 rows=1 loops=60)
        -> Hash Join (cost=389.24..1760.50 rows=592 width=8) (actual time=0.398..6.522 rows=593 loops=60)
          Hash Cond: (b.passenger_id = pd.passenger_id)
          -> Hash Join (cost=150.41..1518.53 rows=1196 width=4) (actual time=0.376..6.389 rows=1200 loops=60)
            Hash Cond: (b.flight_id = f.flight_id)
            -> Seq Scan on booking b (cost=0.00..1179.00 rows=72000 width=8) (actual time=0.001..2.472 rows=72000 loops=60)
            -> Hash (cost=149.00..149.00 rows=113 width=4) (actual time=0.324..0.324 rows=113 loops=60)
              Buckets: 1024 Batches: 1 Memory Usage: 14kB
              -> Seq Scan on flight f (cost=0.00..149.00 rows=113 width=4) (actual time=0.003..0.307 rows=113 loops=60)
                Filter: (airline_id = a.airline_id)
                Rows Removed by Filter: 6687
          -> Hash (cost=195.50..195.50 rows=3466 width=4) (actual time=0.825..0.826 rows=3466 loops=1)
            Buckets: 4096 Batches: 1 Memory Usage: 154kB
            -> Seq Scan on passengerdetails pd (cost=0.00..195.50 rows=3466 width=4) (actual time=0.006..0.516 rows=3466 loops=1)
              Filter: ((sex)::text = 'M'::text)
              Rows Removed by Filter: 3534
        SubPlan 2
          -> Aggregate (cost=1521.52..1521.53 rows=1 width=8) (actual time=6.052..6.052 rows=1 loops=60)
            -> Hash Join (cost=150.41..1518.53 rows=1196 width=8) (actual time=0.363..6.014 rows=1200 loops=60)
              Hash Cond: (b_1.flight_id = f_1.flight_id)
              -> Seq Scan on booking b_1 (cost=0.00..1179.00 rows=72000 width=4) (actual time=0.001..2.342 rows=72000 loops=60)
              -> Hash (cost=149.00..149.00 rows=113 width=4) (actual time=0.313..0.313 rows=113 loops=60)
                Buckets: 1024 Batches: 1 Memory Usage: 14kB
                -> Seq Scan on flight f_1 (cost=0.00..149.00 rows=113 width=4) (actual time=0.003..0.296 rows=113 loops=60)
                  Filter: (airline_id = a.airline_id)
                  Rows Removed by Filter: 6687
            SubPlan 3
              -> Aggregate (cost=1762.86..1762.87 rows=1 width=8) (actual time=6.051..6.051 rows=1 loops=60)
                -> Hash Join (cost=390.09..1761.35 rows=604 width=8) (actual time=0.364..6.033 rows=607 loops=60)
                  Hash Cond: (b_2.passenger_id = pd_1.passenger_id)
                  -> Hash Join (cost=150.41..1518.53 rows=1196 width=4) (actual time=0.342..5.910 rows=1200 loops=60)
                    Hash Cond: (b_2.flight_id = f_2.flight_id)
                    -> Seq Scan on booking b_2 (cost=0.00..1179.00 rows=72000 width=8) (actual time=0.001..2.334 rows=72000 loops=60)
                    -> Hash (cost=149.00..149.00 rows=113 width=4) (actual time=0.292..0.292 rows=113 loops=60)
                      Buckets: 1024 Batches: 1 Memory Usage: 14kB
                      -> Seq Scan on flight f_2 (cost=0.00..149.00 rows=113 width=4) (actual time=0.003..0.278 rows=113 loops=60)
                        Filter: (airline_id = a.airline_id)
                        Rows Removed by Filter: 6687
                  -> Hash (cost=195.50..195.50 rows=3534 width=4) (actual time=0.816..0.816 rows=3534 loops=1)
                    Buckets: 4096 Batches: 1 Memory Usage: 157kB
                    -> Seq Scan on passengerdetails pd_1 (cost=0.00..195.50 rows=3534 width=4) (actual time=0.006..0.519 rows=3534 loops=1)
                      Filter: ((sex)::text = 'F'::text)
                      Rows Removed by Filter: 3466
                SubPlan 4
                  -> Aggregate (cost=1521.52..1521.53 rows=1 width=8) (actual time=6.132..6.132 rows=1 loops=60)
                    -> Hash Join (cost=150.41..1518.53 rows=1196 width=8) (actual time=0.338..6.095 rows=1200 loops=60)
                      Hash Cond: (b_3.flight_id = f_3.flight_id)
                      -> Seq Scan on booking b_3 (cost=0.00..1179.00 rows=72000 width=4) (actual time=0.001..2.375 rows=72000 loops=60)
                      -> Hash (cost=149.00..149.00 rows=113 width=4) (actual time=0.288..0.288 rows=113 loops=60)
                        Buckets: 1024 Batches: 1 Memory Usage: 14kB
                        -> Seq Scan on flight f_3 (cost=0.00..149.00 rows=113 width=4) (actual time=0.004..0.274 rows=113 loops=60)
                          Filter: (airline_id = a.airline_id)
                          Rows Removed by Filter: 6687
          Planning Time: 0.521 ms
          JIT:
            Functions: 82
            Options: Inlining false, Optimization false, Expressions true, Deforming true
            Timing: Generation 2.384 ms, Inlining 0.000 ms, Optimization 0.672 ms, Emission 22.003 ms, Total 25.059 ms
          Execution Time: 1524.928 ms
          (64 rows)

```

	Direct Join	CTE	Subquery
Urutan Pemrosesan Data (Keyword: Join, Group By, SubPlan)	1. Hash Join pada b.flight_id = f.flight_id 2. Hash Join f.airline_id = a.airline_id 3. Hash Join b.passenger_id = pd.passenger_id 4. Group by	1. Hash Join b.flight_id = f.flight_id 2. Hash Join b.passenger_id = pd.passenger_id 3. Group By f.airline_id 4. Scan	Subquery 1: 1. Filter flight airline_id = a.airline_id 2. Hash join b.flight_id = f.flight_id 3. Hash join b.passenger_id = pd.passenger_id

	a.airline_id 5. Order By (Sort) a.airlinenime	Subquery CTE airplane_passenger_summary 5. Hash Join a.airline_id = aps.airline_id (dgn CTE) 6. Order By (Sort) a.airline_name	er_id 4. Count (agregasi) tuple hasil subquery Subquery 2: 1. Filter flight airline_id = a.airline_id 2. Hash Join b_1.flight_id = f_1.flight_id 3. Count (agregasi) tuple hasil subquery Subquery 3: 1. Filter flight airline_id = a.airline_id Hash Join 2. Select pd.sex = 'F' 3. Hash join b_2.flight_id = f_2.flight_id 4. Hash join b_2.pasenger_id = f_2.pasenger_id 5. Count tuple hasil subquery Subquery 4: 1. Filter flight airline_id = a.airline_id Hash Join 2. Hash Join b_3.flight_id = f_3.flight_id 3. Count tuple hasil subquery Order By (Sort) on a.airlinename
Efisiensi Waktu (Tinggi/Sedang/Rendah, kenapa?)	Sedang, query ini menggunakan join langsung sehingga optimizer bisa menentukan jenis join/urutan yang paling optimal. Tapi pada query ini memang	Sedang. Mirip seperti join langsung, optimizer masih memiliki fleksibilitas dalam menentukan query plan yang paling optimal sehingga	Paling rendah, karena ini menggunakan correlated subquery yang artinya optimizer tidak bisa flexibel memilih plan yang

	membutuhkan JOIN banyak tabel.	time dan cost juga mirip dengan direct join.	paling efisien (seperti join/urutan join)
Dampak pada PostgreSQL <=11 (beri “-” jika tidak ada)	-	Tidak dieksekusi secara inline	-
Readability & Maintainability (Sangat Baik/Baik/Buruk)	Readability & Maintainability bagus jika tabel masih kecil atau jumlah tabel yg dibutuhkan masih dalam skala kecil.	Readability bagus, karena dia menggunakan CTE yang bisa dipakai berulang.	Maintainability buruk, karena semakin besar jumlah data maka subquery akan dijalankan pada jumlah data yang sangat besar.

2. Fast Pace Query

- a. Jelaskan perbedaan Sequential Scan dan Index Scan (jawab dengan singkat, padat, dan jelas)

Sequential scan melakukan scan pada tiap tuple untuk mencari syarat yang memenuhi, sementara Index scan melakukan lookup pada tabel Index untuk mencari Index yang memenuhi lalu mengambil data sesuai pointer.

- b. Jelaskan perbedaan antara EXPLAIN dan EXPLAIN ANALYZE (jawab dengan singkat, padat, dan jelas)

EXPLAIN hanya menampilkan query plan yang diperkirakan oleh optimizer tanpa benar-benar menjalankan query. Sementara EXPLAIN ANALYZE menampilkan query plan dan estimasi dengan menjalankan query.

- c. Jelaskan kondisi yang ideal untuk query plan Nested Loop, Hash, dan Merge Join (jawab dengan singkat, padat, dan jelas)

Nested Loop : Ideal jika tabel outer lebih kecil atau tabel inner memiliki Index.
Hash : Ideal digunakan pada equi join dan tabel tidak harus terurut.
Merge Join: Ideal pada tabel yang sudah terurut karena hanya harus scan masing-masing tabel sekali untuk join.

Coups bekerja sebagai database administrator pada sistem informasi bandara. Atasannya meminta Coups mengambil data penerbangan seorang penumpang berdasarkan nomor pasport dengan query sebagai berikut.

```

SELECT
    b.booking_id,
    b.seat,
    b.price,
    p.firstname,
    p.lastname,
    p.passportno,
    f.flightno,
    f.departure,
    f.arrival,
    a.airlinename,
    ap.identifier AS airplane_type
FROM
    booking b
JOIN
    passenger p ON b.passenger_id = p.passenger_id
JOIN
    flight f ON b.flight_id = f.flight_id
JOIN
    airline a ON f.airline_id = a.airline_id
JOIN
    airplane ar ON f.airplane_id = ar.airplane_id
JOIN
    airplane_type ap ON ar.type_id = ap.type_id
WHERE
    f.departure = '2025-05-26 10:21:00'
    AND ar.airline_id = 17
    AND p.passportno = 'B3113948';

```

Namun setelah dilakukan pengujian, query tersebut membutuhkan waktu cukup lama sehingga Coups ingin melihat Query Plan dari query yang dijalankan.

Query Masukan	<pre> EXPLAIN ANALYZE SELECT b.booking_id, b.seat, b.price, p.firstname, p.lastname, p.passportno, f.flightno, f.departure, f.arrival, a.airlinename, ap.identifier AS airplane_type FROM booking b JOIN passenger p ON b.passenger_id = p.passenger_id JOIN flight f ON b.flight_id = f.flight_id JOIN airline a ON f.airline_id = a.airline_id JOIN airplane ar ON f.airplane_id = ar.airplane_id JOIN airplane_type ap ON ar.type_id = ap.type_id WHERE f.departure = '2025-05-26 10:21:00' </pre>
-----------------------------	---

	AND ar.airline_id = 17 AND p.passportno = 'B3113948';
SS Query Plan	
<pre> QUERY PLAN ----- Nested Loop (cost=138.94..1526.51 rows=1 width=288) (actual time=6.767..7.074 rows=1 loops=1) -> Nested Loop (cost=138.79..1521.65 rows=1 width=74) (actual time=6.747..7.053 rows=1 loops=1) Join Filter: (ar.airplane_id = f.airplane_id) Rows Removed by Join Filter: 4 -> Nested Loop (cost=138.79..1512.07 rows=1 width=74) (actual time=6.710..7.016 rows=1 loops=1) Join Filter: (a.airline_id = f.airline_id) Rows Removed by Join Filter: 27 -> Nested Loop (cost=138.79..1509.72 rows=1 width=65) (actual time=6.705..7.011 rows=1 loops=1) -> Hash Join (cost=138.51..1506.62 rows=10 width=38) (actual time=1.145..6.951 rows=8 loops=1) Hash Cond: (b.passenger_id = p.passenger_id) -> Seq Scan on booking b (cost=0.00..1179.00 rows=72000 width=21) (actual time=0.004..2.633 rows=72000 loops=1) -> Hash (cost=138.50..138.50 rows=1 width=25) (actual time=0.536..0.537 rows=1 loops=1) Buckets: 1024 Batches: 1 Memory Usage: 9kB -> Seq Scan on passenger p (cost=0.00..138.50 rows=1 width=25) (actual time=0.025..0.531 rows=1 loops=1) Filter: ((passportno)::text = 'B3113948'::text) Rows Removed by Filter: 6999 -> Index Scan using flight_pkey on flight f (cost=0.28..0.31 rows=1 width=35) (actual time=0.007..0.007 rows=0 loops=8) Index Cond: (flight_id = b.flight_id) Filter: (departure = '2025-05-26 10:21:00'::timestamp without time zone) Rows Removed by Filter: 1 -> Seq Scan on airline a (cost=0.00..1.60 rows=60 width=17) (actual time=0.002..0.002 rows=28 loops=1) -> Seq Scan on airplane ar (cost=0.00..9.50 rows=6 width=8) (actual time=0.029..0.036 rows=5 loops=1) Filter: (airline_id = 17) Rows Removed by Filter: 413 -> Index Scan using airplane_type_pkey on airplane_type ap (cost=0.15..4.83 rows=1 width=222) (actual time=0.020..0.020 rows=1 loops=1) Index Cond: (type_id = ar.type_id) Planning Time: 1.101 ms Execution Time: 7.121 ms (28 rows) Time: 9.010 ms </pre>	

Untuk meningkatkan performa query, Coups mencoba menambahkan index pada kolom *passportno* pada tabel *passenger*.

Query Perbaikan yang dilakukan Coups
<pre> CREATE INDEX idx_passenger_passportno ON passenger(passportno); </pre>
SS Query Perbaikan
<pre> airport=# CREATE INDEX idx_passenger_passportno ON passenger(passportno); CREATE INDEX Time: 14.740 ms </pre>
SS Query Plan setelah Perbaikan

```

QUERY PLAN
-----
Nested Loop (cost=8.60..1407.75 rows=1 width=288) (actual time=6.276..6.595 rows=1 loops=1)
  Join Filter: (ap.type_id = ar.type_id)
  Rows Removed by Join Filter: 35
  -> Nested Loop (cost=8.60..1391.45 rows=1 width=74) (actual time=6.261..6.580 rows=1 loops=1)
    Join Filter: (ar.airplane_id = f.airplane_id)
    Rows Removed by Join Filter: 4
    -> Nested Loop (cost=8.60..1381.87 rows=1 width=74) (actual time=6.238..6.557 rows=1 loops=1)
      Join Filter: (a.airline_id = f.airline_id)
      Rows Removed by Join Filter: 27
      -> Nested Loop (cost=8.60..1379.52 rows=1 width=65) (actual time=6.233..6.552 rows=1 loops=1)
        -> Hash Join (cost=8.31..1376.42 rows=10 width=38) (actual time=0.507..6.518 rows=8 loops=1)
          Hash Cond: (b.passenger_id = p.passenger_id)
          -> Seq Scan on booking b (cost=0.00..1179.00 rows=72000 width=21) (actual time=0.003..2.700 rows=72000 loops=1)
          -> Hash (cost=8.30..8.30 rows=1 width=25) (actual time=0.029..0.030 rows=1 loops=1)
            Buckets: 1024 Batches: 1 Memory Usage: 9kB
            -> Index Scan using idx_passenger_passportno on passenger p (cost=0.28..8.30 rows=1 width=25) (actual time=0.023..0.024 rows=1 loops=1)
              Index Cond: ((passportno)::text = 'B3113948'::text)
        -> Index Scan using flight_pkey on flight f (cost=0.28..0.31 rows=1 width=35) (actual time=0.004..0.004 rows=0 loops=8)
          Index Cond: (flight_id = b.flight_id)
          Filter: (departure = '2025-05-26 10:21:00'::timestamp without time zone)
          Rows Removed by Filter: 1
      -> Seq Scan on airline a (cost=0.00..1.60 rows=60 width=17) (actual time=0.003..0.003 rows=28 loops=1)
    -> Seq Scan on airplane ar (cost=0.00..9.50 rows=6 width=8) (actual time=0.014..0.022 rows=5 loops=1)
      Filter: (airline_id = 17)
      Rows Removed by Filter: 413
  -> Seq Scan on airplane_type ap (cost=0.00..12.80 rows=280 width=222) (actual time=0.011..0.012 rows=36 loops=1)
Planning Time: 0.719 ms
Execution Time: 6.658 ms
(28 rows)

```

Lihat *query plan* setelah penambahan indeks untuk mengetahui apakah terjadi perubahan atau tidak. (Jawab salah satu)

- Jika terjadi perubahan, tunjukkan bagian yang berubah dan jelaskan alasan perubahan tersebut.

Ya, terjadi perubahan.

- Planning time menjadi lebih singkat awalnya planning time 1.101ms, setelah ditambahkan index menjadi 0.719ms
- Cost menurun cukup signifikan. Pada query awal, costnya adalah 138.94...1526.51 namun setelah ditambahkan index costnya menjadi 8.60...1407.75
- Execution time menurun dari yang awalnya 7.121ms menjadi 6.658ms
- Pada query awal masih menggunakan sequential scan yang mahal, karena dia ngescan seluruh baris. Namun setelah ditambahkan index, scan dilakukan dengan index scan.

```

-> Seq Scan on passenger p (cost=0.00..138.50 rows=1 width=25) (actual time=0.025..0.531 rows=1 loops=1)
  Filter: ((passportno)::text = 'B3113948'::text)
  Rows Removed by Filter: 6999

-> Index Scan using idx_passenger_passportno on passenger p (cost=0.28..8.30 rows=1 width=25) (actual time=0.023..0.024 rows=1 loops=1)
  Index Cond: ((passportno)::text = 'B3113948'::text)

```

- Jika tidak terjadi perubahan, jelaskan mengapa tidak ada perubahan.

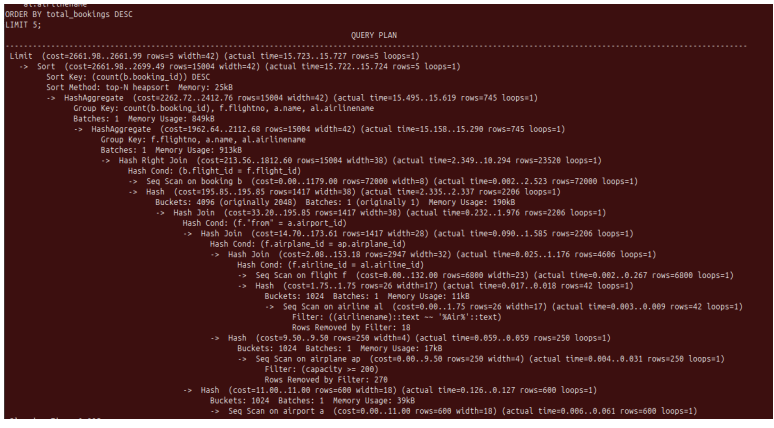
-

3. Query Kok Gini?

Zoho meminta Yeho untuk mencari 5 penerbangan dengan jumlah booking terbanyak. Penerbangan yang dicari harus menggunakan pesawat dengan kapasitas minimal 200 penumpang dan dioperasikan oleh maskapai yang nama maskapainya (*airlinename*) **diawali dengan kata "Air"**. Hasil yang ditampilkan cukup berupa **nomor penerbangan, nama maskapai, dan jumlah booking**.

Yeho telah menulis query berikut untuk menjawab permintaan tersebut:

Query
<pre> SELECT DISTINCT f.flightno, a.name AS departure_airport, al.airlinename AS airline, COUNT(b.booking_id) AS total_bookings FROM flight AS f JOIN airport AS a ON f."from" = a.airport_id JOIN airline AS al ON f.airline_id = al.airline_id JOIN airplane AS ap ON f.airplane_id = ap.airplane_id LEFT JOIN booking AS b ON f.flight_id = b.flight_id WHERE al.airlinename LIKE '%Air%' AND ap.capacity >= 200 GROUP BY f.flightno, a.name, al.airlinename ORDER BY total_bookings DESC LIMIT 5; </pre>

	SS Query
Tampilkan dalam 2 SS terpisah: - Query Plan - Bagian Time	

	<pre> Planning Time: 0.995 ms Execution Time: 16.363 ms (37 rows) Time: 18.395 ms </pre>
--	---

Menurut Yeho, query tersebut sudah dapat menghasilkan informasi yang dibutuhkan. Namun, Zoho menemukan bahwa masih terdapat beberapa bagian dari query yang **kurang tepat dan belum optimal**, sehingga terdapat pekerjaan tambahan yang sebenarnya tidak diperlukan dalam proses pencarian data.

Bantulah Yeho menganalisis query tersebut dari aspek ketepatan dan efisiensi waktu serta biaya (time and cost). Identifikasi minimal 3 kelemahan utama yang terdapat pada query tersebut beserta solusi untuk memperbaikinya.

Notes:

Praktikan tidak perlu menuliskan kembali *query* yang telah diperbaiki. Cukup berikan penjelasan singkat mengenai kelemahan dan usulan perbaikannya. Pastikan setiap usulan perbaikan tetap menghasilkan makna semantik yang sesuai dengan kebutuhan awal.

Kelemahan 1
Filtering maskapai berawalan kata “Air” pada query WHERE al.airlinename LIKE '%Air%' menggunakan wildcard (%) pada kedua ujung, padahal seharusnya % diletakkan di akhir saja. Karena wildcard diletakkan di kedua ujung, optimizer tidak dapat menerapkan indexing, dan akhirnya harus dilakukan pattern-matching ke seluruh tuple pada relasi. Namun, setelah diperbaiki menjadi “Air%”, jika DBMS mendukung, maka dapat dilakukan indexing.
Solusi Singkat 1
Ubah menjadi WHERE al.airlinename LIKE 'Air%'

Kelemahan 2
Ada agregasi COUNT(b.booking_id) dengan cost yang sangat besar, yaitu sebesar 2262.72..2412.76
Solusi Singkat 2
Membuat materialized view untuk menyimpan hasil COUNT

Kelemahan 3
Terdapat banyak join pada query ini sehingga membutuhkan cost yang besar

Solusi Singkat 3

Membuat View untuk menyimpan hasil join antar tabel agar query bisa dijalankan lebih cepat

4. Wong Senpai

Mr Boss puas dengan kinerja Wong selama magang, sehingga Wong mendapatkan return offer sebagai karyawan tetap. Salah satu tugas barunya adalah membimbing dan memantau kinerja tiga intern baru: Aday, Inmay, dan Laif.

Teman Mr Boss, Mr Best, memiliki jaringan toko batik tulis di bandara-bandara di seluruh Indonesia. Menjelang Hari Kemerdekaan Indonesia pada 17 Agustus, Mr Best ingin mengirimkan promosi kepada **penumpang asing (penumpang yang negara asalnya bukan Indonesia)** yang akan **tiba di atau berangkat dari Indonesia pada bulan Agustus 2025**. Karena sebagian besar pembeli batik tulis berasal dari kalangan atas, promosi hanya akan dikirimkan kepada penumpang kelas atas, yaitu **penumpang yang membayar lebih dari 1,5 kali rata-rata harga tiket pada penerbangan yang sama**. Mr Best hanya membutuhkan **nama depan, nama belakang, email, negara asal, dan jenis kelamin**.

Mr Best meminta bantuan Mr Boss untuk mendapatkan daftar kontak penumpang tersebut, dan Mr Boss menyerahkan tugas ini kepada ketiga intern. Berikut adalah query yang mereka buat:

Query yang dilakukan Para Intern

```
SELECT DISTINCT *
FROM booking b
JOIN passenger p          ON p.passenger_id = b.passenger_id
JOIN passengerdetails pd  ON pd.passenger_id = p.passenger_id
JOIN flight f             ON f.flight_id = b.flight_id
JOIN airline al           ON al.airline_id = f.airline_id
JOIN airport_geo dep      ON dep.airport_id = f."from"
JOIN airport_geo arr      ON arr.airport_id = f."to"
WHERE NOT (UPPER(pd.country) = 'INDONESIA')
      AND (UPPER(dep.country) = 'INDONESIA' OR UPPER(arr.country) =
'INDONESIA')
      AND f.departure::date BETWEEN '2025-08-01' AND '2025-08-31'
      AND b.price > (
      SELECT AVG(b2.price) * 1.5
      FROM booking b2
      WHERE b2.flight_id = f.flight_id
      )
ORDER BY b.price DESC;
```

SS Eksekusi Query Para Intern (Aktifkan \timing)

```
airport=# SELECT DISTINCT *
FROM booking b
JOIN passenger p      ON p.passenger_id = b.passenger_id
JOIN passengerdetails pd ON pd.passenger_id = p.passenger_id
JOIN flight f         ON f.flight_id = b.flight_id
JOIN airline al       ON al.airline_id = f.airline_id
JOIN airport_geo dep   ON dep.airport_id = f."from"
JOIN airport_geo arr   ON arr.airport_id = f."to"
WHERE NOT (UPPER(pd.country) = 'INDONESIA')
      AND (UPPER(dep.country) = 'INDONESIA' OR UPPER(arr.country) = 'INDONESIA')
      AND f.departure::date BETWEEN '2025-08-01' AND '2025-08-31'
      AND b.price > (
        SELECT AVG(b2.price) * 1.5
        FROM booking b2
        WHERE b2.flight_id = f.flight_id
      )
ORDER BY b.price DESC;
Time: 7961.871 ms (00:07.962)
```

SS Pemrosesan Query Para Intern (dengan EXPLAIN ANALYZE)

```
QUERY PLAN
-----
Unique  (cost=7893.85..7893.15 rows=1 width=138) (actual time=7679.346..7679.367 rows=28 loops=1)
  -> Sort (cost=7893.85..7893.86 rows=1 width=138) (actual time=7679.363..7679.378 rows=28 loops=1)
    Sort Key: b.price DESC, b.booking_id, b.flight_id, b.seat, b.passenger_id, p.passportno, p.firstname, pd.birthday, pd.sex, pd.street, pd.city, pd.zip, pd.country, pd.emailaddress, pd.telephonenumber, f.flightno, f."from", f."to", f.departure, f.arrival, f.airline_id, f.airplane_id, al.airline_id, al.base_airport, dep.name, dep.city, dep.country, dep.latitude, dep.longitude, dep.geolocation, arr.name, arr.city, arr.country, arr.latitude, arr.longitude, arr.geolocation
    Sort Method: quicksort  Memory: 3348
  -> Nested Loop (cost=223.78..7892.52 rows=1 width=138) (actual time=243.877..7679.251 rows=28 loops=1)
    -> Nested Loop (cost=221.40..7892.22 rows=1 width=138) (actual time=243.853..7679.181 rows=28 loops=1)
      -> Hash Join (cost=221.71..7891.98 rows=1 width=138) (actual time=243.833..7679.889 rows=28 loops=1)
        Hash Cond: (f."to" = arr.airport_id)
        Join Filter: (upper(dep.country)::text = 'INDONESIA'::text) OR (upper(arr.country)::text = 'INDONESIA'::text)
        Rows Removed by Join Filter: 95
        -> Nested Loop (cost=208.71..7889.89 rows=128 width=141) (actual time=70.159..7678.912 rows=117 loops=1)
          -> Hash Join (cost=208.43..7884.65 rows=128 width=141) (actual time=70.132..7678.494 rows=117 loops=1)
            Hash Cond: (b.flight_id = f.flight_id)
            Join Filter: (b.price > SubPlan 1)
            Rows Removed by Join Filter: 3558
            -> Seq Scan on booking b (cost=0.00..1179.00 rows=72880 width=21) (actual time=0.802..2.473 rows=72880 loops=1)
            -> Hash (cost=208.00..208.00 rows=34 width=43) (actual time=0.700..0.700 rows=34 loops=1)
              Buckets: 1024  Batches: 1  Memory Usage: 2048
              -> Seq Scan on flight f (cost=0.00..208.00 rows=34 width=43) (actual time=0.004..0.665 rows=34 loops=1)
                Filter: (((departure::date <= '2025-08-01'::date) AND ((departure::date <= '2025-08-31'::date)))
                Rows Removed by Filter: 8466
              SubPlan 1
                -> Aggregate (cost=1359.83..1359.84 rows=1 width=32) (actual time=2.087..2.087 rows=1 loops=3675)
                  -> Seq Scan on booking b2 (cost=0.00..1359.00 rows=1 width=6) (actual time=1.048..2.085 rows=13 loops=3675)
                    Filter: (flight_id = f.flight_id)
                    Rows Removed by Filter: 71987
                  -> Memoize (cost=2.29..1.83 rows=1 width=77) (actual time=0.802..0.802 rows=1 loops=117)
                    Cache Key: f."from"
                    Cache Mode: logical
                    Hits: 43  Misses: 74  Evictions: 0  Overflows: 0  Memory Usage: 1348
                    -> Index Scan using airport_geo_idx on airport_geo dep (cost=0.28..1.82 rows=1 width=77) (actual time=0.802..0.802 rows=1 loops=74)
                      Index Cond: (airport_id = f."from")
                    -> Hash (cost=15.00..15.00 rows=600 width=77) (actual time=0.104..0.104 rows=600 loops=1)
                      Buckets: 1024  Batches: 1  Memory Usage: 640
                      -> Seq Scan on airport_geo arr (cost=0.00..15.00 rows=600 width=77) (actual time=0.004..0.037 rows=600 loops=1)
                        Index Cond: (passenger_id = b.passenger_id)
                    -> Index Scan using passengerdetails_idx on passengerdetails pd (cost=0.28..0.31 rows=1 width=91) (actual time=0.003..0.003 rows=1 loops=22)
                      Filter: (upper(country)::text = 'INDONESIA'::text)
                      Rows Removed by Filter: 8
                    -> Index Scan using passenger_idx on passenger p (cost=0.28..0.30 rows=1 width=25) (actual time=0.002..0.002 rows=1 loops=28)
                      Index Cond: (passenger_id = b.passenger_id)
                    -> Index Scan using airline_idx on airline al (cost=0.14..0.51 rows=1 width=24) (actual time=0.001..0.001 rows=1 loops=28)
                      Index Cond: (airline_id = f.airline_id)
Planning Time: 1.464 ms
Execution Time: 7679.572 ms
(6 rows)
```

Catatan: Praktikan perlu mengeksekusi query di atas terlebih dahulu sebelum melanjutkan soal!

Saat meninjau hasil kerja mereka, Wong menyadari bahwa query tersebut berjalan sangat lambat dan ditulis dengan cara yang tidak efisien. Bantulah Wong untuk:

1. Menganalisis penyebab query tersebut berjalan lambat
2. Memperbaiki query tersebut agar berjalan lebih cepat dan menghasilkan data yang dibutuhkan Mr Best.
3. Menunjukkan bukti perbaikan dengan membandingkan hasil **EXPLAIN ANALYZE** sebelum dan sesudah perbaikan.

Catatan penting: Perbaikan hanya boleh dilakukan dengan menulis ulang query. Praktikan tidak diperbolehkan membuat index atau mengubah struktur tabel.

Query Perbaikan	<pre> WITH more_than_avg AS (SELECT b.price FROM booking b JOIN flight f ON b.flight_id = f.flight_id JOIN booking b2 ON b2.flight_id = f.flight_id GROUP BY b.price HAVING (b.price > AVG(b2.price) * 1.5)) SELECT p.firstname, p.lastname, pd.emailaddress, pd.country, pd.sex FROM passenger p JOIN passengerdetails pd ON pd.passenger_id = p.passenger_id JOIN booking b ON p.passenger_id = b.passenger_id JOIN flight f ON f.flight_id = b.flight_id JOIN booking b2 ON b2.flight_id = f.flight_id JOIN airline al ON al.airline_id = f.airline_id JOIN airport_geo dep ON dep.airport_id = f."from" JOIN airport_geo arr ON arr.airport_id = f."to" WHERE NOT (pd.country = 'Indonesia') AND (dep.country = 'Indonesia' OR arr.country = 'Indonesia') AND f.departure::date BETWEEN '2025-08-01' AND '2025-08-31' AND b.price IN (SELECT price FROM more_than_avg) ORDER BY b.price DESC; </pre>
-----------------	--

SS Eksekusi Query Perbaikan (Aktifkan \timing)	
airport=#	<pre> WITH more_than_avg AS(SELECT b.price FROM booking b JOIN flight f ON b.flight_id = f.flight_id JOIN booking b2 ON b2.flight_id = f.flight_id GROUP BY b.price HAVING (b.price > AVG(b2.price) * 1.5)) SELECT p.firstname, p.lastname, pd.emailaddress, pd.country, pd.sex FROM passenger p JOIN passengerdetails pd ON pd.passenger_id = p.passenger_id JOIN booking b ON p.passenger_id = b.passenger_id JOIN flight f ON f.flight_id = b.flight_id JOIN booking b2 ON b2.flight_id = f.flight_id JOIN airline al ON al.airline_id = f.airline_id JOIN airport_geo dep ON dep.airport_id = f."from" JOIN airport_geo arr ON arr.airport_id = f."to" WHERE NOT (pd.country = 'Indonesia') AND (dep.country = 'Indonesia' OR arr.country = 'Indonesia') AND f.departure::date BETWEEN '2025-08-01' AND '2025-08-31' AND b.price IN (SELECT price FROM more_than_avg) ORDER BY b.price DESC; Time: 352.184 ms airport=# </pre>

SS Pemrosesan Query Perbaikan Anda (dengan EXPLAIN ANALYZE)

```

QUERY PLAN
Sort (cost=73051.08..73052.68 rows=282 width=56) (actual time=380.798..380.880 rows=148 loops=1)
Sort Key: b.price DESC
Sort Method: quicksort Memory: 37kB
-> Hash Join (cost=64527.77..73050.42 rows=282 width=56) (actual time=259.544..380.750 rows=148 loops=1)
Hash Cond: (b.flight_id = f.flight_id)
-> HashAggregate (cost=61295.89..69744.11 rows=1912 width=6) (actual time=236.695..369.544 rows=2154 loops=1)
Group Key: b.flight_id
Filter: (b.flight_id = (avg(b2.flight_id) * 1.5))
Planned Partitions: 4 Batches: 5 Memory Usage: 29089kB
Rows Removed by Filter: 59685
-> Hash Join (cost=2485.12..12457.65 rows=76253 width=12) (actual time=18.729..82.393 rows=912514 loops=1)
Hash Cond: (b.flight_id = f.flight_id)
-> Seq Scan on booking b1 (cost=0.00..1179.00 rows=72000 width=10) (actual time=0.002..3.896 rows=72000 loops=1)
-> Hash (cost=1585.12..1585.12 rows=72000 width=10) (actual time=18.463..18.464 rows=72000 loops=1)
Buckets: 131072 Batches: 5 Memory Usage: 4399kB
-> Hash Join (cost=217.00..1585.12 rows=72000 width=14) (actual time=0.909..11.440 rows=72000 loops=1)
Hash Cond: (b2.flight_id = f.flight_id)
-> Seq Scan on booking b2 (cost=0.00..1179.00 rows=72000 width=10) (actual time=0.001..2.431 rows=72000 loops=1)
-> Hash (cost=132.00..132.00 rows=6800 width=4) (actual time=0.886..0.886 rows=6800 loops=1)
Buckets: 8192 Batches: 1 Memory Usage: 384kB
-> Seq Scan on flight f1 (cost=0.00..132.00 rows=6800 width=4) (actual time=0.001..0.369 rows=6800 loops=1)
-> Hash (cost=3221.24..3221.24 rows=847 width=56) (actual time=19.860..19.865 rows=18151 loops=1)
Buckets: 16384 (originally 1824) Batches: 1 (originally 3) Memory Usage: 1946kB
-> Hash Join (cost=1673.77..3221.24 rows=847 width=56) (actual time=18.875..17.828 rows=10351 loops=1)
Hash Cond: (b2.flight_id = f.flight_id)
-> Seq Scan on booking b2 (cost=0.00..1179.00 rows=72000 width=10) (actual time=0.005..2.525 rows=72000 loops=1)
-> Hash (cost=1672.77..1672.77 rows=88 width=64) (actual time=10.859..10.863 rows=814 loops=1)
Buckets: 1824 Batches: 1 Memory Usage: 848kB
-> Hash Join (cost=248.34..1672.77 rows=88 width=64) (actual time=1.033..10.675 rows=814 loops=1)
Hash Cond: (f.airline_id = a.airline_id)
-> Nested Loop (cost=245.99..1679.50 rows=88 width=68) (actual time=0.996..18.520 rows=814 loops=1)
Join Filter: (p.passenger_id = pd.passenger_id)
-> Nested Loop (cost=245.43..1615.45 rows=95 width=22) (actual time=0.997..9.560 rows=814 loops=1)
Join Filter: ((dep.country)::text = 'Indonesia')::text OR ((arr.country)::text = 'Indonesia')::text)
-> Hash Join (cost=222.93..1592.00 rows=368 width=34) (actual time=0.789..7.822 rows=3675 loops=1)
Hash Cond: (f.flight_id = dep.airport_id)
-> Hash Join (cost=200.43..1568.55 rows=368 width=30) (actual time=0.668..7.302 rows=3675 loops=1)
Hash Cond: (b.flight_id = f.flight_id)
-> Seq Scan on booking b (cost=0.00..1179.00 rows=72000 width=14) (actual time=0.001..2.555 rows=72000 loops=1)
-> Hash (cost=200.00..200.00 rows=34 width=16) (actual time=0.659..0.659 rows=334 loops=1)
Buckets: 1824 Batches: 1 Memory Usage: 20kB
-> Seq Scan on flight f (cost=0.00..200.00 rows=34 width=16) (actual time=0.004..0.615 rows=334 loops=1)
Filter: (((departure)::date <= '2025-08-01'::date) AND ((departure)::date <= '2025-08-31'::date))
Rows Removed by Filter: 6466
-> Hash (cost=15.00..15.00 rows=680 width=12) (actual time=0.113..0.113 rows=680 loops=1)
Buckets: 1824 Batches: 1 Memory Usage: 35kB
-> Seq Scan on airport geo (cost=0.00..15.00 rows=680 width=12) (actual time=0.002..0.039 rows=680 loops=1)
-> Hash (cost=15.00..15.00 rows=680 width=12) (actual time=0.177..0.177 rows=680 loops=1)
Buckets: 1824 Batches: 1 Memory Usage: 35kB
-> Seq Scan on airport geo arr (cost=0.00..15.00 rows=680 width=12) (actual time=0.004..0.070 rows=680 loops=1)
-> Index Scan using passengerdetails_pk on passengerdetails pd (cost=0.28..0.31 rows=1 width=42) (actual time=0.001..0.001 rows=1 loops=992)
Index Cond: (passenger_id = b.passenger_id)
Filter: ((country)::text <= 'Indonesia')::text
Rows Removed by Filter: 0
-> Index Scan using passenger_pk on passenger p (cost=0.28..0.30 rows=1 width=16) (actual time=0.001..0.001 rows=1 loops=814)
Index Cond: (passenger_id = b.passenger_id)

```

Penjelasan dan Justifikasi

Terdapat beberapa kesalahan pada query yang menyebabkan eksekusi query berjalan secara lambat.

- Informasi yang sebenarnya dibutuhkan adalah nama depan, nama belakang, email, negara asal, dan jenis kelamin; tetapi query melakukan SELECT seluruh atribut dengan SELECT *
 - PERBAIKAN: SELECT atribut yang diperlukan saja
- Query menggunakan correlated subquery AND b.price > (SELECT AVG(b2.price) * 1.5 FROM booking b2 WHERE b2.flight_id = f.flight_id). Artinya, subquery ini akan dijalankan secara terpisah pada setiap baris di relasi, sehingga menyebabkan penalti performa.
 - PERBAIKAN: correlated subquery dapat dijadikan CTE yang akan dieksekusi secara inline bersama query (jika menggunakan PostgreSQL >= v.12)
- Optimizer tidak dapat menggunakan index pada atribut dalam function. Pada query, digunakan fungsi UPPER() pada atribut country untuk selection with equality negara Indonesia. Kecuali jika optimizer menggunakan function indexing. Negara seharusnya memiliki penamaan yang tetap dan tidak berbeda-beda, maka
 - PERBAIKAN: Filtering tanpa fungsi UPPER, jadi langsung NOT((pd.country = 'Indonesia') AND (dep.country = 'Indonesia' OR arr.country='Indonesia'))

Setelah diperbaiki, dapat dilihat waktu untuk mengeksekusi query menjadi jauh lebih singkat. Awalnya 7961.871ms, namun dengan query perbaikan waktunya menjadi 352.184ms

5. AirRush

Seorang analis basis data bernama Wowi dengan aura +9999 point bekerja di sebuah perusahaan layanan penerbangan. Suatu hari, seseorang bernama Titi dari tim marketing dengan yang diam-diam ia sukai mengeluh karena dashboard pemesanan tiket berjalan lambat. Dashboard tersebut menampilkan 10 kombinasi maskapai dan negara bandara keberangkatan dengan total nilai pemesanan terbesar.

Setelah diselidiki, query yang digunakan ternyata merupakan query buatan Wowi sendiri:

```
SELECT
    al.airline_id,
    al.airlinename,
    ag.country,
    SUM(b.price) AS total_revenue
FROM booking b
JOIN flight f ON b.flight_id = f.flight_id
JOIN airline al ON f.airline_id = al.airline_id
JOIN airport a ON f."from" = a.airport_id
JOIN airport_geo ag ON a.airport_id = ag.airport_id
GROUP BY al.airline_id, al.airlinename, ag.country
ORDER BY total_revenue DESC, al.airline_id, ag.country
LIMIT 10;
```

SS Eksekusi Query Wowi dengan Timing

```
airport=# SELECT
    al.airline_id,
    al.airlinename,
    ag.country,
    SUM(b.price) AS total_revenue
FROM booking b
JOIN flight f ON b.flight_id = f.flight_id
JOIN airline al ON f.airline_id = al.airline_id
JOIN airport a ON f."from" = a.airport_id
JOIN airport_geo ag ON a.airport_id = ag.airport_id
GROUP BY al.airline_id, al.airlinename, ag.country
ORDER BY total_revenue DESC, al.airline_id, ag.country
LIMIT 10;
 airline_id |   airlinename   | country | total_revenue
-----|-----|-----|-----
          9 | AirAsia         | Indonesia | 1269258.13
         23 | Emirates       | India    | 1196068.66
         19 | EVA Air        | China   | 1029158.38
         40 | Iberia         | France  | 993766.92
          7 | Singapore Airlines | Indonesia | 970381.85
         15 | Asiana Airlines | Japan   | 928706.45
         13 | All Nippon Airways | Hong Kong | 859102.07
         30 | Swiss Intl Air Lines | Germany | 854090.37
         51 | Avianca        | Finland | 818320.81
          2 | Citilink       | Indonesia | 811659.12
(10 rows)

Time: 36.838 ms
airport=#
```

SS Eksekusi Query Arturo dengan Query Plan

```
airport=# EXPLAIN ANALYZE
airport=# SELECT
  al.airline_id,
  al.airlinename,
  ag.country,
  SUM(b.price) AS total_revenue
FROM booking b
JOIN flight f ON b.flight_id = f.flight_id
JOIN airline al ON f.airline_id = al.airline_id
JOIN airport a ON f."from" = a.airport_id
JOIN airport_geo ag ON a.airport_id = ag.airport_id
GROUP BY al.airline_id, al.airlinename, ag.country
ORDER BY total_revenue DESC, al.airline_id, ag.country
LIMIT 10;

QUERY PLAN
-----
Limit (cost=2853.82..2853.85 rows=10 width=57) (actual time=48.492..48.496 rows=10 loops=1)
-> Sort (cost=2853.82..2861.32 rows=3000 width=57) (actual time=48.491..48.495 rows=10 loops=1)
    Sort Key: (sum(b.price)) DESC, al.airline_id, ag.country
    Sort Method: top-N heapsort  Memory: 26kB
    -> HashAggregate (cost=2751.49..2788.99 rows=3000 width=57) (actual time=48.281..48.383 rows=560 loops=1)
        Group Key: al.airline_id, ag.country
        Batches: 1  Memory Usage: 360kB
        -> Hash Join (cost=260.35..2211.49 rows=72000 width=31) (actual time=1.484..37.039 rows=72000 loops=1)
            Hash Cond: (f."from" = ag.airport_id)
            -> Hash Join (cost=237.85..1998.66 rows=72000 width=31) (actual time=1.342..29.676 rows=72000 loops=1)
                Hash Cond: (f."from" = a.airport_id)
                -> Hash Join (cost=219.35..1789.82 rows=72000 width=27) (actual time=1.248..22.211 rows=72000 loops=1)
                    Hash Cond: (f.airline_id = al.airline_id)
                    -> Hash Join (cost=217.00..1585.12 rows=72000 width=14) (actual time=1.227..14.860 rows=72000 loops=1)
                        Hash Cond: (b.flight_id = f.flight_id)
                        -> Seq Scan on booking b (cost=0.00..1179.00 rows=72000 width=10) (actual time=0.002..3.184 rows=72000 loops=1)
                        -> Hash (cost=122.00..132.00 rows=6000 width=12) (actual time=1.204..1.204 rows=6000 loops=1)
                            Buckets: 8192  Batches: 1  Memory Usage: 357kB
                            -> Seq Scan on flight f (cost=0.00..122.00 rows=6000 width=12) (actual time=0.001..0.530 rows=6000 loops=1)
                        -> Hash (cost=1.60..1.60 rows=60 width=17) (actual time=0.014..0.014 rows=60 loops=1)
                            Buckets: 1024  Batches: 1  Memory Usage: 11kB
                            -> Seq Scan on airline al (cost=0.00..1.60 rows=60 width=17) (actual time=0.001..0.005 rows=60 loops=1)
                        -> Hash (cost=11.00..11.00 rows=600 width=4) (actual time=0.084..0.085 rows=600 loops=1)
                            Buckets: 1024  Batches: 1  Memory Usage: 36kB
                            -> Seq Scan on airport a (cost=0.00..11.00 rows=600 width=4) (actual time=0.006..0.038 rows=600 loops=1)
                        -> Hash (cost=15.00..15.00 rows=600 width=12) (actual time=0.137..0.139 rows=600 loops=1)
                            Buckets: 1024  Batches: 1  Memory Usage: 36kB
                            -> Seq Scan on airport_geo ag (cost=0.00..15.00 rows=600 width=12) (actual time=0.005..0.055 rows=600 loops=1)
            Planning Time: 1.140 ms
            Execution Time: 48.656 ms
            (30 rows)
        Time: 50.339 ms
```

Hasil Analisis

Terdapat banyak join, sehingga dihasilkan biaya yang sangat besar dan memakan waktu yang banyak untuk agregasi (2751.49..2788.99), group by, dan sorting (2853.82..2861..32).

- Wowi mengetahui bahwa laporan sering dibaca, sedangkan perubahan data pemesanan relatif jarang terjadi. Ia memutuskan menggunakan materialized view untuk menyimpan hasil perhitungan.

Buat materialized view bernama `mv_airline_country_revenue` yang menyimpan **hasil agregasi seluruh kombinasi maskapai dan negara keberangkatan**, tanpa LIMIT. Kemudian, buat query pembacaan yang menghasilkan laporan yang sama dengan query awal.

Query perbaikan

```
CREATE materialized view mv_airline_country_revenue AS
SELECT
  al.airline_id,
  al.airlinename,
  ag.country,
  SUM(b.price) AS total_revenue
FROM booking b
JOIN flight f ON b.flight_id = f.flight_id
JOIN airline al ON f.airline_id = al.airline_id
JOIN airport a ON f."from" = a.airport_id
JOIN airport_geo ag ON a.airport_id = ag.airport_id
```

	GROUP BY al.airline_id, al.airlinename, ag.country ORDER BY total_revenue DESC, al.airline_id, ag.country;
SS Pembuatan View	
<pre>airport=# CREATE materialized view mv_airline_country_revenue AS SELECT al.airline_id, al.airlinename, ag.country, SUM(b.price) AS total_revenue FROM booking b JOIN flight f ON b.flight_id = f.flight_id JOIN airline al ON f.airline_id = al.airline_id JOIN airport a ON f."from" = a.airport_id JOIN airport_geo ag ON a.airport_id = ag.airport_id GROUP BY al.airline_id, al.airlinename, ag.country; ORDER BY total_revenue DESC, al.airline_id, ag.country; SELECT 568; Time: 41.217 ms</pre>	
Query select	SELECT * FROM mv_airline_country_revenue LIMIT 10;
SS Hasil Eksekusi Query dengan Timing setelah pembuatan View	
<pre>airport=# SELECT * FROM mv_airline_country_revenue LIMIT 10; airline_id airlineame country total_revenue ----- 9 AirAsia Indonesia 1269258.13 23 Emirates India 1196068.66 19 EVA Air China 1029158.38 40 Iberia France 993766.92 7 Singapore Airlines Indonesia 978381.85 15 Asiana Airlines Japan 928786.45 13 All Nippon Airways Hong Kong 859102.07 30 Swiss Intl Air Lines Germany 854090.37 51 Avianca Finland 818320.01 2 Citilink Indonesia 811659.12 (10 rows) Time: 0.731 ms airport=#</pre>	

- b. Setelah materialized view dibuat, Wowi dengan auranya menyadari bahwa perubahan data pemesanan tidak otomatis muncul pada laporan.

Buat fungsi trigger dan trigger untuk menjalankan **REFRESH MATERIALIZED VIEW** setelah operasi pada tabel booking. Pastikan refresh dijalankan sekali untuk setiap perintah perubahan.

Asumsikan hanya tabel booking yang berubah selama pengujian dan tidak terdapat perubahan dari sesi lain.

Query Trigger	
SS Hasil Eksekusi Pembuatan Trigger	

Query Pengujian Trigger	<pre>BEGIN; -- sebelum perubahan SELECT SUM(total_revenue) AS total_sebelum FROM mv_airline_country_revenue; -- Ubah satu pemesanan yang ada dalam laporan UPDATE booking</pre>
-------------------------	---

	<pre> SET price = price + 100 WHERE booking_id = (SELECT b.booking_id FROM booking b JOIN flight f ON b.flight_id = f.flight_id JOIN airline al ON f.airline_id = al.airline_id JOIN airport a ON f."from" = a.airport_id JOIN airport_geo ag ON a.airport_id = ag.airport_id WHERE b.price IS NOT NULL ORDER BY b.booking_id LIMIT 1); -- Total setelah refresh oleh trigger SELECT SUM(total_revenue) AS total_setelah FROM mv_airline_country_revenue; ROLLBACK; </pre>
SS Hasil Pengujian Trigger	

Kesimpulan dan penjelasan (jelaskan selengkap mungkin)
Query tersebut memiliki cost dan waktu eksekusi yang besar karena banyaknya join, adanya agregasi, group by, dan sort. Maka, karena query hanya sering dibaca dan jarang diubah, maka materialized view cocok karena menyimpan hasil agregasi dan join disimpan sehingga saat ingin baca data cost dan time menjadi sangat kecil.
Bonus: Wowi kemudian bertanya “Hmm Jika tujuannya menyimpan hasil query, apa bedanya menggunakan materialized view dengan membuat tabel biasa menggunakan CREATE TABLE ... AS SELECT ...?” Jelaskan perbedaannya dari sisi penyimpanan hasil, pembaruan data, dan kemudahan pemeliharaan. Dari sisi akses, lebih cepat mengakses dari materialized view, dan tidak diperlukan query yang complex. Namun, pada tabel biasa, akses bisa jadi lebih lambat tergantung pada query yang ditulis dan skema tabel. Dari sisi pemeliharaan dan pembaruan data, materialized view harus di-refresh berkala agar tetap up-to-date dengan tabel biasa. Dari sisi penyimpanan hasil, materialized view hanya bisa diakses (tidak dapat melakukan penulisan), berbeda dengan tabel biasa dimana operasi read/write didukung. Selain itu, pada materialized view, tidak dapat ditambahkan primary key (PK) ataupun foreign key (FK).

6. Penghuni Terminal 67

Saat pesawat Viktor Navorski masih di udara, terjadi kudeta di Krakozhia, negara asalnya. Begitu mendarat, paspornya tidak diakui siapa pun. Tidak bisa masuk, tidak bisa pulang, jadi ia tinggal di terminal.

Sembilan bulan di *gate* 67 membuat orang melakukan hal-hal aneh. Viktor mulai iseng menulis *query* ke basis data bandara, lalu berhenti pada satu pertanyaan sederhana: berapa total pendapatan tiap pesawat?

Query 1

```
SELECT a.airplane_id,
       (SELECT SUM(b.price)
        FROM booking b
        JOIN flight f ON b.flight_id = f.flight_id
        WHERE f.airplane_id = a.airplane_id) AS
total_revenue
FROM airplane a;
```

Query 2

```
SELECT a.airplane_id, SUM(b.price) AS total_revenue
FROM airplane a
JOIN airplane_type t ON t.type_id = a.type_id
JOIN airline al      ON al.airline_id = a.airline_id
LEFT JOIN flight f   ON f.airplane_id = a.airplane_id
LEFT JOIN booking b  ON b.flight_id = f.flight_id
GROUP BY a.airplane_id;
```

Query 3

```
WITH airplane_revenue AS (
  SELECT f.airplane_id, SUM(b.price) AS total
  FROM booking b
  JOIN flight f ON b.flight_id = f.flight_id
  GROUP BY f.airplane_id
)
SELECT a.airplane_id, ar.total AS total_revenue
FROM airplane a
LEFT JOIN airplane_revenue ar ON ar.airplane_id =
a.airplane_id;
```

la penasaran dengan kinerja masing-masing *query* dalam segi evaluasi biaya dan waktu eksekusi. Bantulah Viktor mengevaluasi kinerja masing-masing *query* dan bandingkan ketiganya dalam segi evaluasi biaya dan waktu eksekusi! Jelaskan apa yang mempengaruhi cepat/lambatnya kinerja masing-masing *query*. Gunakan keyword EXPLAIN ANALYZE dan pastikan *timing* sudah teraktivasi.

Hasil Evaluasi Query 1

```

QUERY PLAN
-----
Seq Scan on airplane a  (cost=0.00..789182.23 rows=520 width=36) (actual time=78.588..2828.725 rows=520 loops=1)
  SubPlan 1
    -> Aggregate  (cost=1517.63..1517.64 rows=1 width=32) (actual time=5.299..5.299 rows=1 loops=520)
      -> Hash Join  (cost=149.16..1517.28 rows=138 width=6) (actual time=0.572..5.284 rows=138 loops=520)
        Hash Cond: (b.flight_id = f.flight_id)
        -> Seq Scan on booking b  (cost=0.00..1179.00 rows=72000 width=10) (actual time=0.001..2.319 rows=72000 loops=520)
        -> Hash  (cost=149.00..149.00 rows=13 width=4) (actual time=0.179..0.179 rows=13 loops=520)
          Buckets: 1024 Batches: 1 Memory Usage: 9kB
          -> Seq Scan on flight f  (cost=0.00..149.00 rows=13 width=4) (actual time=0.014..0.172 rows=13 loops=520)
            Filter: (airplane_id = a.airplane_id)
            Rows Removed by Filter: 6787
Planning Time: 0.226 ms
JIT:
  Functions: 18
  Options: Inlining true, Optimization true, Expressions true, Deforming true
  Timing: Generation 0.646 ms, Inlining 8.004 ms, Optimization 37.529 ms, Emission 27.470 ms, Total 73.650 ms
Execution Time: 2829.469 ms
(17 rows)

Time: 2830.191 ms (00:02.830)

```

Hasil Evaluasi Query 2

```

airport=# EXPLAIN ANALYZE SELECT a.airplane_id, SUM(b.price) AS total_revenue
FROM airplane a
JOIN airplane_type t ON t.type_id = a.type_id
JOIN airline al ON al.airline_id = a.airline_id
LEFT JOIN flight f ON f.airplane_id = a.airplane_id
LEFT JOIN booking b ON b.flight_id = f.flight_id
GROUP BY a.airplane_id;
QUERY PLAN
-----
HashAggregate  (cost=2563.22..2569.72 rows=520 width=36) (actual time=43.241..43.320 rows=520 loops=1)
  Group Key: a.airplane_id
  Batches: 1 Memory Usage: 297kB
  -> Hash Join  (cost=250.35..2203.22 rows=72000 width=10) (actual time=1.381..34.764 rows=72000 loops=1)
    Hash Cond: (a.airline_id = al.airline_id)
    -> Hash Join  (cost=248.00..1990.52 rows=72000 width=14) (actual time=1.361..28.448 rows=72000 loops=1)
      Hash Cond: (a.type_id = t.type_id)
      -> Hash Right Join  (cost=231.70..1790.36 rows=72000 width=10) (actual time=1.342..21.690 rows=72000 loops=1)
        Hash Cond: (f.airplane_id = a.airplane_id)
        -> Hash Right Join  (cost=217.00..1585.12 rows=72000 width=10) (actual time=1.252..13.478 rows=72000 loops=1)
          Hash Cond: (b.flight_id = f.flight_id)
          -> Seq Scan on booking b  (cost=0.00..1179.00 rows=72000 width=10) (actual time=0.002..3.237 rows=72000 loops=1)
          -> Hash  (cost=132.00..132.00 rows=6800 width=8) (actual time=1.221..1.221 rows=6800 loops=1)
            Buckets: 8192 Batches: 1 Memory Usage: 330kB
            -> Seq Scan on flight f  (cost=0.00..132.00 rows=6800 width=8) (actual time=0.002..0.585 rows=6800 loops=1)
        -> Hash  (cost=8.20..8.20 rows=520 width=12) (actual time=0.086..0.086 rows=520 loops=1)
          Buckets: 1024 Batches: 1 Memory Usage: 31kB
          -> Seq Scan on airplane a  (cost=0.00..8.20 rows=520 width=12) (actual time=0.004..0.036 rows=520 loops=1)
      -> Hash  (cost=12.80..12.80 rows=280 width=4) (actual time=0.011..0.012 rows=37 loops=1)
        Buckets: 1024 Batches: 1 Memory Usage: 10kB
        -> Seq Scan on airplane_type t  (cost=0.00..12.80 rows=280 width=4) (actual time=0.003..0.005 rows=37 loops=1)
    -> Hash  (cost=1.00..1.60 rows=60 width=4) (actual time=0.018..0.018 rows=60 loops=1)
      Buckets: 1024 Batches: 1 Memory Usage: 11kB
      -> Seq Scan on airline al  (cost=0.00..1.60 rows=60 width=4) (actual time=0.004..0.007 rows=60 loops=1)
Planning Time: 0.359 ms
Execution Time: 43.463 ms
(26 rows)

Time: 44.445 ms
airport=#

```

Hasil Evaluasi Query 3

```
airport=# EXPLAIN ANALYZE WITH airplane_revenue AS (
  SELECT f.airplane_id, SUM(b.price) AS total
  FROM booking b
  JOIN flight f ON b.flight_id = f.flight_id
  GROUP BY f.airplane_id
)
SELECT a.airplane_id, ar.total AS total_revenue
FROM airplane a
LEFT JOIN airplane_revenue ar ON ar.airplane_id = a.airplane_id;
QUERY PLAN
-----
Hash Right Join  (cost=1959.82..1972.90 rows=520 width=36) (actual time=20.121..20.255 rows=520 loops=1)
  Hash Cond: (f.airplane_id = a.airplane_id)
    -> HashAggregate (cost=1945.12..1951.62 rows=520 width=36) (actual time=19.993..20.074 rows=520 loops=1)
      Group Key: f.airplane_id
      Batches: 1  Memory Usage: 297kB
      -> Hash Join  (cost=217.00..1585.12 rows=72000 width=10) (actual time=1.166..12.011 rows=72000 loops=1)
        Hash Cond: (b.flight_id = f.flight_id)
          -> Seq Scan on booking b  (cost=0.00..1179.00 rows=72000 width=10) (actual time=0.002..2.549 rows=72000 loops=1)
          -> Hash  (cost=132.00..132.00 rows=6800 width=8) (actual time=1.142..1.142 rows=6800 loops=1)
            Buckets: 8192  Batches: 1  Memory Usage: 330kB
            -> Seq Scan on flight f  (cost=0.00..132.00 rows=6800 width=8) (actual time=0.002..0.533 rows=6800 loops=1)
        -> Hash  (cost=8.20..8.20 rows=520 width=4) (actual time=0.125..0.125 rows=520 loops=1)
          Buckets: 1024  Batches: 1  Memory Usage: 27kB
          -> Seq Scan on airplane a  (cost=0.00..8.20 rows=520 width=4) (actual time=0.007..0.034 rows=520 loops=1)
    Planning Time: 0.238 ms
    Execution Time: 20.374 ms
(16 rows)

Time: 21.068 ms
airport=#
```

Penjelasan dan Perbandingan Hasil Evaluasi

Query 1:

Secara execution time dan cost jauh paling besar dibandingkan 2 query lainnya. Query ini menggunakan correlated subquery, sehingga beberapa proses harus diulang berkali-kali (jln yg banyak) ini bisa dilihat dari banyaknya loop pada Scan on flight f, Scan on booking, dan Hash Join keduanya. Ini karena optimizer tidak bisa lebih bebas menentukan algoritma / plan join yang lebih efisien pada correlated subquery.

Query 2:

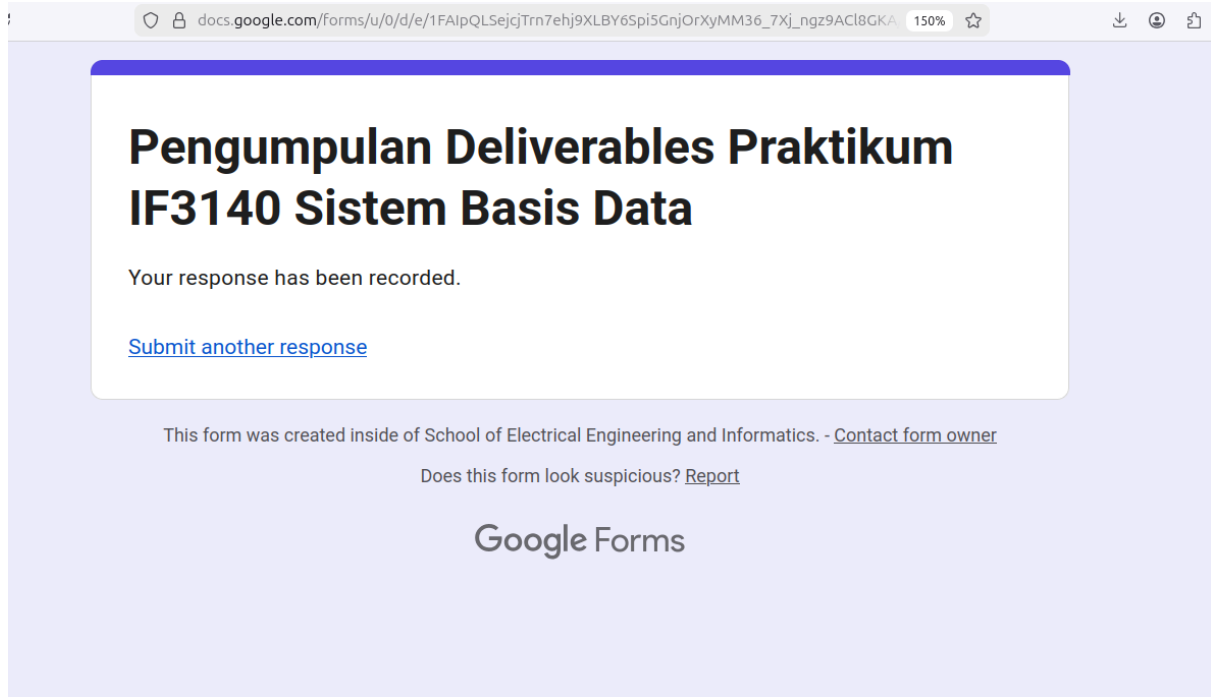
Cost dan execution time pada query 2 ini lumayan besar karena banyaknya jumlah proses join dan di paling akhir harus melakukan agregasi pada jumlah rows yang banyak sehingga cost & time untuk sum juga besar.

Query 3:

Paling kecil secara execution time dan cost karena memanfaatkan CTE untuk agregat sum sehingga bisa meminimalkan cost agregat yang merupakan operasi yang mahal pada query ini jadi bisa menurunkan overall cost query.

Pembagian Kerja

NIM	Tugas
13524089	2,4,6
13524098	1,3,5



The screenshot shows a Google Form titled "Pengumpulan Deliverables Praktikum IF3140 Sistem Basis Data". The form is displayed in a browser window with the URL "docs.google.com/forms/u/0/d/e/1FAIpQLSejcjTrn7ehj9XLBY6Spi5GnjOrXyMM36_7Xj_ngz9ACl8GKA". The form content includes a confirmation message: "Your response has been recorded." and a link to "Submit another response". At the bottom, it states "This form was created inside of School of Electrical Engineering and Informatics. - Contact form owner" and "Does this form look suspicious? Report". The Google Forms logo is also visible.

Kita sudah kumpul kakak-kakak!!! Terima kasih semangat terus!!!